Mandatory vs Recommended Labeling Policy Design — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Mandatory vs Recommended Labeling Policy Design (Microsoft Purview) 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

What is delivered . . . 5

Business outcome . . . 5

Important boundaries . . . 6

1. Solution Overview — Mandatory vs Recommended Labeling Policy Design 6

What this solution is . . . 6

Why this matters . . . 6

The four policy levers (what this solution configures) . . . 7

Labeling decision matrix (the design starting point — confirm with your governance owners) . . . 7

Naming convention (used in every doc and script) . . . 8

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Mandatory vs Recommended Labeling Policy Design 9

2.1 A published sensitivity-label taxonomy already exists . . . 9

2.2 Licensing for labeling and recommended (client-side) labeling . . . 10

2.3 Information Protection role (least privilege) . . . 10

2.4 Security & Compliance PowerShell connectivity . . . 10

2.5 A pilot ring and a pilot-policy commitment . . . 11

2.6 An agreed default classification and downgrade rule . . . 11

2.7 Per-app friction decisions (documents vs Outlook) . . . 11

Global Requirements — Microsoft Purview Solutions 12

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Mandatory vs Recommended Labeling Policy Design 14

Step 1 — Assess the current label-policy posture (Assess) . . . 14

Step 2 — Design the mandatory-vs-recommended matrix (Design) . . . 14

Step 3 — Create (or select) the pilot label policy (Configure) . . . 15
Step 4 — Set the default label on the pilot policy (Configure, -WhatIf first) . . . 15
Step 5 — Enable mandatory labeling on the pilot policy (Configure, -WhatIf first) . . . 15
Step 6 — Require downgrade justification on the pilot policy (Configure, -WhatIf first) . . . 16
Step 7 — Configure recommended-label prompts on the label (Configure, -WhatIf first) . . . 16
Step 8 — Validate friction and effective settings on the pilot (Validate) . . . 16
Step 9 — Expand the settings to the broad policy (Expand) . . . 16
Step 10 — Produce the posture evidence and operate (Validate → Operate) . . . 17
5. Testing & Validation Guide — Mandatory vs Recommended Labeling Policy Design 17
Test 1 — Effective policy settings match the design . . . 17
Test 2 — Default label applies to new content (pilot) . . . 17
Test 3 — Mandatory labeling prompts on documents . . . 18
Test 4 — Outlook friction matches the per-app decision . . . 18
Test 5 — Downgrade requires a justification and records it . . . 18
Test 6 — Downgrade justification appears in audit (evidence) . . . 18
Test 7 — Recommended prompt suggests (does not force) the label . . . 18
Test 8 — No conflicting settings across overlapping policies . . . 19
Test 9 — Broad-policy settings match the accepted pilot design . . . 19
Test 10 — Posture evidence is complete . . . 19
6. Troubleshooting Guide — Mandatory vs Recommended Labeling Policy Design (Microsoft Purview) 19
A. Availability / setup . . . 20
B. Behaviour doesn't apply (false negatives) . . . 20
C. Behaviour is too aggressive (false positives / adoption pain) . . . 21
D. Downgrade justification . . . 21
E. Conflicting / overlapping policies . . . 21
F. Expansion / evidence . . . 22
Escalation checklist . . . 22
7. Best Practices — Mandatory vs Recommended Labeling Policy Design 22
Choosing mandatory vs recommended . . . 22
Rollout discipline (no simulation — so pilot is everything) . . . 23
Defaults, downgrade, and evidence . . . 23
Avoiding policy conflicts . . . 23
Program & change management . . . 24
Security & safety hygiene . . . 24
8. Deliverables Summary — Mandatory vs Recommended Labeling Policy Design 24
What you receive . . . 24

Documentation 24
PowerShell tools (5) 25
Outcomes 26
Explicitly out of scope (deferred) 26
Appendix A — PowerShell Scripts 28

Executive Summary — Mandatory vs Recommended Labeling Policy Design (Microsoft Purview)

The problem

A sensitivity-label taxonomy only protects content that actually gets labelled — and it is the label **policy**, not the label, that decides application rates. Organisations get this wrong in both directions. Mandate labels everywhere and users click the first (lowest) label just to dismiss the prompt, corrupting the classification data and flooding the help desk; mandate nothing and content stays unlabelled, so DLP, encryption, and retention have nothing to key off. Left unaddressed, this is a compliance gap and a data-risk exposure.

The solution

A complete, self-guided design and configuration of the **label-policy behaviour** that balances the two: **mandatory labeling** where the risk justifies friction, **recommended (prompt) labeling** where a nudge is enough, a sensible **default label** so nothing is silently unclassified, and **downgrade justification** for an audit trail. It decides *where to require a label versus suggest one* — a mandatory-vs-recommended matrix — and configures each lever, piloted on a ring and validated before it touches everyone. It uses only the Microsoft Purview portal and Security & Compliance PowerShell — no third-party tooling.

How it works

The engagement follows one disciplined model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Assess the current label-policy posture; design a signed-off mandatory-vs-recommended matrix (defaults, downgrade rules, prompts); configure the settings on a **pilot policy scoped to a pilot ring**; validate the *friction* (ticket volume, send-blocking, mislabel rate) as well as the effective settings; then expand the accepted settings to the broad policy. Because label-policy settings have **no simulation mode**, the pilot ring is the safety mechanism — every write is previewed with `-WhatIf` first. Five PowerShell tools — read-only or `-WhatIf` by default — automate discovery, configuration, effective-settings validation, posture reporting, and a policy-conflict health check.

What is delivered

- Six implementation documents (Overview, Requirements, Implementation, Testing, Best Practices, Troubleshooting) plus this summary.
- Five Security & Compliance PowerShell tools: posture assessment, a configuration/remediation helper (mandatory/default/downgrade/recommend), effective-settings validation, posture-matrix reporting, and a policy-conflict health helper.
- A named, gated, reversible rollout with explicit pass/fail criteria at every step.

Business outcome

High classification rates **without a user revolt** — labels required where they matter, suggested where a nudge suffices, defaulted so nothing escapes unlabelled, and downgrades evidenced. The policy design is

configured, piloted, validated, and defensible — at a fraction of the cost of a bespoke consulting engagement.

Important boundaries

Which populations *should* be mandated, the enterprise default classification, and the downgrade rules are **determinations owned by your compliance, security, and records functions** — this solution provides the technical configuration method and evidence, not the determination. The label taxonomy, encryption, and retention are assumed to exist (separate solutions). Recommended (client-side) labeling with advanced conditions is **licensing-dependent** (E5 / E5 Compliance) and the Purview portal is evolving — availability, portal paths, and advanced-setting keys are flagged "verify your licensing/tenant," never assumed.

1. Solution Overview — Mandatory vs Recommended Labeling Policy Design

What this solution is

Once a sensitivity-label taxonomy exists, the **label policy** decides *how hard* the tenant asks users to classify: whether a label is **required** (mandatory), **suggested** (recommended, via a prompt), or **pre-set** (a default label), and whether removing or lowering a label demands a **justification**. This solution is the **policy-design** build for exactly those behaviours — it does not create the label taxonomy or encryption (that is a separate solution); it decides and configures **where to require a label versus where to merely suggest one**, sets sensible **defaults**, turns on **downgrade justification**, and configures **recommended-label prompts** — all piloted on a small ring first, validated, then expanded, using only the Microsoft Purview portal and Security & Compliance PowerShell.

Over-mandating labels tanks adoption (users click the first label to get past the prompt, or file a ticket); under-mandating leaves content unclassified, so DLP, encryption, and retention have nothing to key off. This solution is the disciplined middle: **mandatory where the risk justifies friction, recommended where a nudge is enough, defaults so nothing is unlabelled by accident.**

Why this matters

- **Labels are only useful if they get applied.** A perfect taxonomy that nobody applies protects nothing. The *policy* — not the label — determines application rates.
- **Mandatory labeling forces a decision** at save/send time. Powerful, but blunt: applied too broadly it trains users to pick any label to dismiss the prompt, which corrupts your classification data.
- **A default label guarantees a floor.** Every new document/email starts at a known classification (e.g. `General`) instead of "no label", so content is never silently unclassified.
- **Downgrade justification creates evidence.** When a user removes a label or lowers `Confidential` → `Public`, requiring a typed justification both deters careless downgrades and produces an **audit trail** for investigations.
- **Recommended labeling nudges without blocking.** A *recommend* prompt ("This looks like it contains financial data — apply **Confidential**?") raises correct-label rates without the backlash of hard

enforcement.

The four policy levers (what this solution configures)

Lever	What it does	Where it is set	Blunt or soft?
Mandatory labeling	User must apply *a* label before they can save a document or send an email	Label **policy** setting ("require users to apply a label"), per-app override via advanced settings	Blunt (hard stop)
Default label	A label is pre-selected on new documents/email (and optionally sites/groups, Power BI)	Label **policy** setting ("apply a default label"), per-app default via advanced settings	Soft (floor)
Downgrade justification	User must type a justification to remove a label or lower its classification	Label **policy** setting ("require justification…")	Soft (evidence + friction)
Recommended labeling	A label is *suggested* with a prompt/policy-tip when content matches conditions — user accepts or dismisses	The **label's** auto-labeling for Office apps, set to **Recommend** (not Automatic)	Soft (nudge)

> ***Mandatory vs Recommended is the core decision.*** *Mandatory forces *some* label but not the *right* one; Recommended suggests the *right* label but the user can decline. The design skill is deciding, per population and per content type, which lever (or combination) fits — see the decision matrix below.*

Labeling decision matrix (the design starting point — confirm with your governance owners)

> *A **technical starting point**; the final decisions (which populations are mandated, the default label, downgrade rules) are an information-governance determination — confirm with the accountable owner. Names here are reused verbatim across the docs and scripts.*

Population / content	Mandatory?	Default label	Downgrade justification?	Recommended prompts?
Pilot ring (`grp-labeling-pilot@contoso.com`)	Yes (validate friction here first)	`General`	Yes	Yes — on `Confidential \ All Employees`
General workforce (after pilot)	Yes (documents + email)	`General`	Yes	Yes

Population / content	Mandatory?	Default label	Downgrade justification?	Recommended prompts?
Front-line / high-volume senders	Consider **recommend-only** to protect throughput	`General`	Yes	Yes
Outlook (email)	Often **relaxed** vs documents (`DisableMandatoryInOutlook`) to avoid send-blocking	`General` (Outlook default)	Yes	Yes
External-share-heavy sites	Default label + recommend	`General`	Yes	Yes

Naming convention (used in every doc and script)

Every artifact this solution creates or configures is named and reused verbatim across the docs, scripts, and portal:

Artifact	Name
Label policy (pilot)	`LabelPolicy – Pilot – Ring1`
Label policy (broad)	`LabelPolicy – Broad – AllUsers`
Pilot ring group	`grp-labeling-pilot@contoso.com`
Default label (documents & email)	`General`
Recommended-prompt label	`Confidential \ All Employees`
Recommended-prompt message	*"This content looks sensitive — apply Confidential to protect it."*

What you can control

• **Which populations are mandated** to apply a label, and on which apps (documents vs Outlook can differ).

• **The default label** for documents and email (and, where licensed, sites/groups, Power BI, meetings).

• **Whether downgrade/removal requires a justification**, and the justification prompt.

• **Which labels recommend themselves** (with a policy-tip message) when content matches conditions — a nudge, not a block.

• **Rollout order** — pilot ring → broad — and rollback (unpublish/relax a setting) per policy.

Limitations (be honest with buyers)

• **These are label *policy* settings, not a simulation feature.** Unlike service-side auto-labeling, mandatory/default/downgrade settings have **no "simulation" mode** — the safe rollout mechanism is a

pilot policy scoped to a pilot ring (validate friction there), then broad. This solution enforces pilot-first for exactly that reason.

• **Recommend vs Automatic is a per-label setting.** Recommended labeling is configured on the **label's** auto-labeling for Office apps (set to *Recommend*), which is **client-side** and **licensing-dependent** for some conditions (advanced classifiers) — **verify your licensing/tenant**.

• **Advanced-setting keys and portal paths evolve.** Per-app overrides (e.g. `Mandatory`, `DisableMandatoryInOutlook`, `OutlookDefaultLabel`) are documented advanced settings, but Microsoft's portal and some keys change — each is flagged "verify in your tenant"; none is invented.

• **The taxonomy itself is out of scope.** This solution assumes labels already exist (or references the taxonomy/labels solution); it designs the *policy behaviour* over them.

• **Determinations are deferred.** What *should* be mandated, the enterprise default classification, and records/legal implications are **governance determinations** — this solution provides the configuration method and evidence.

The standard deployment model

> *Assess the current label-policy posture → design the mandatory-vs-recommended matrix (defaults, downgrade rules, prompts) → configure on a **pilot policy scoped to a pilot ring** (`-WhatIf` → commit) → validate friction and effective settings → **expand** the settings to the broad policy → validate → operate. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Mandatory vs Recommended Labeling Policy Design

Verify **every** requirement below before starting the Implementation Guide. These are the **solution-specific** prerequisites for designing and configuring label-policy behaviour (mandatory, default, downgrade justification, recommended prompts); they build on the shared **Global Requirements — Microsoft Purview Solutions** (licensing, least-privilege roles, Security & Compliance PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, and a pilot scope with change control). Read the Global Requirements first; this guide does not repeat them.

> ***Determination note.*** **Which* populations must be mandated, *what* the enterprise default classification should be, and the downgrade rules are **information-governance determinations** owned by your compliance/security/records stakeholders — not technical defaults. This solution provides the **configuration method and evidence**; confirm the policy intent with the accountable owner before enforcing.*

2.1 A published sensitivity-label taxonomy already exists

• **Description:** This solution designs the **policy behaviour** over labels; the **labels themselves** (e.g. `Public`, `General`, `Confidential \ All Employees`, `Highly Confidential`) must already be created and available. If they do not exist, build them first (the taxonomy/labels-implementation solution) — you cannot set a default label or a recommended prompt for a label that does not exist.

• **Why required:** `Set-LabelPolicy` default-label and the recommend prompt reference specific labels by name/GUID.

• **How to verify:** `Get-Label | Format-Table DisplayName,Name,Priority` returns the intended taxonomy; the labels appear in Purview → **Information Protection** → **Sensitivity labels**.

• **How to implement if missing:** Create the taxonomy first; return here once labels exist and are (or are about to be) published.

2.2 Licensing for labeling and recommended (client-side) labeling

• **Description:** Manual sensitivity labeling, mandatory labeling, default labels, and downgrade justification are available broadly (Microsoft 365 **E3** tier covers manual labeling and label policies). **Recommended / automatic client-side labeling in Office apps** — including the conditions that trigger a recommend prompt — requires **Microsoft 365 E5** or the **E5 Compliance** add-on for the advanced classifiers, though basic sensitive-info-type-based recommendations are broadly available. **Verify your licensing/tenant** — Microsoft's licensing tables are the authority.

• **Why required:** The recommended-labeling step will not work as designed without the plan the condition type requires.

• **How to verify:** M365 admin center → subscriptions/add-ons; confirm the label's *Auto-labeling for Office apps* page is available in the Purview portal.

• **How to implement if missing:** Assign/purchase the required plan for the pilot users before configuring recommended prompts; mandatory/default/downgrade still work at E3.

2.3 Information Protection role (least privilege)

• **Description:** The Purview role group this solution needs is **Information Protection Admin** (create/manage sensitivity **label policies** and the labels' auto-labeling settings). Global Administrator is **not** required for label-policy work.

• **Why required:** `Set-LabelPolicy`, `New-LabelPolicy`, and the label auto-labeling cmdlets/settings are gated by this role group.

• **How to verify:** Purview portal → **Roles & scopes** → **Permissions**; confirm the admin account is in *Information Protection* (or *Information Protection Admin*).

• **How to implement if missing:** Add the admin to the least-privilege role group under change control; wait for role propagation before running cmdlets.

2.4 Security & Compliance PowerShell connectivity

• **Description:** A workstation with the **ExchangeOnlineManagement** module (which provides `Connect-IPPSSession` for Security & Compliance PowerShell), with a modern-auth / MFA-capable admin account.

• **Why required:** All five provided scripts and every configuration cmdlet run through `Connect-IPPSSession`.

• **How to verify:** `Connect-IPPSSession` succeeds and `Get-LabelPolicy` returns without error.

• **How to implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm modern-auth sign-in. See the Global Requirements for the full toolchain.

2.5 A pilot ring and a pilot-policy commitment

• **Description:** A named **pilot ring** (a small group of users — the group backing `LabelPolicy – Pilot – Ring1`) to receive the mandatory/default/downgrade/recommend settings **first**, and an explicit commitment that these settings are validated on the pilot policy before they are applied to `LabelPolicy – Broad – AllUsers`. Because label-policy settings have **no simulation mode**, the pilot ring **is** the safety mechanism.

• **Why required:** Mandatory labeling and downgrade justification change every user's save/send experience; the pilot surfaces friction (send-blocking, ticket volume) before it hits everyone.

• **How to verify:** Confirm the pilot group membership, a change window, and an approver.

• **How to implement if missing:** Create the pilot group and agree the change-control path before configuring anything.

2.6 An agreed default classification and downgrade rule

• **Description:** A decision, from your governance owners, on the **enterprise default label** (the classification every new document/email should start at — commonly `General`, never a protected/encrypting label) and whether **downgrade/removal requires a justification** (and what the justification prompt should say).

• **Why required:** The default label is a real business decision — defaulting too high causes over-classification and encryption surprises; too low defeats the purpose. `Set-LabelPolicy` needs a specific label as the default.

• **How to verify:** Confirm the named default label and the downgrade decision with the accountable owner; confirm the default label is **non-encrypting** unless explicitly intended.

• **How to implement if missing:** Get the default-classification and downgrade decisions signed off before configuring the policy.

2.7 Per-app friction decisions (documents vs Outlook)

• **Description:** A decision on whether mandatory labeling applies **equally to documents and Outlook**, or whether **Outlook is relaxed** (many organisations disable *mandatory-in-Outlook* to avoid blocking email send, using the documented `DisableMandatoryInOutlook` advanced setting), and whether Outlook has its own **default label** (`OutlookDefaultLabel`).

• **Why required:** Blocking email send on every message drives the loudest adoption backlash; the per-app split is a common, deliberate design choice that this solution configures.

• **How to verify:** Confirm the documents-vs-Outlook mandatory decision and any Outlook-specific default with stakeholders.

• **How to implement if missing:** Decide the per-app behaviour before applying advanced settings; default to piloting mandatory on **documents** first, Outlook relaxed, then tighten if needed.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Mandatory vs Recommended Labeling Policy Design

Follow these steps **in order**. Configuration is done in the **Microsoft Purview portal** (`https://purview.microsoft.com`) and **Security & Compliance PowerShell** (`Connect-IPPSSession`). Every write is run `-WhatIf` first, and every behaviour is **piloted on a policy scoped to the pilot ring** before it touches the broad policy — label-policy settings have **no simulation mode**, so the pilot ring is the safety mechanism. Artifact names follow the convention from the Overview and are reused verbatim. Portal paths are current at authoring — **verify in your tenant**, as the portal is evolving; where an advanced-setting key is version-sensitive it is flagged.

> ***Model:** Assess the posture → design the mandatory-vs-recommended matrix → configure on the **pilot policy** (`-WhatIf` → commit) → validate friction + effective settings → **expand** to the broad policy → validate. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand. Determination deferral:** which populations are mandated, the default classification, and the downgrade rule are **governance determinations** — confirm each before enforcing (see §2).*

Step 1 — Assess the current label-policy posture (Assess)

1. Run `IPMANREC-Discovery.ps1` (read-only). It inventories existing **label policies** (`Get-LabelPolicy`), each policy's **advanced settings** (`Mandatory`, `DisableMandatoryInOutlook`, `OutlookDefaultLabel`, default-label configuration), the policy **scope** (which groups/locations it targets), and each **label's auto-labeling for Office apps** mode (Automatic vs Recommend) via `Get-Label`.

2. Review the output: is any policy already mandating labels? Is a default label already set? Do any labels already recommend themselves? This is your baseline — **do not** create conflicting settings across overlapping policies.

3. **Gate:** you have a written baseline of every label policy, its advanced settings, its scope, and each label's recommend/automatic state, and you have confirmed no conflicting mandatory/default settings across policies that both target the same users.

Step 2 — Design the mandatory-vs-recommended matrix (Design)

1. With your governance owners, finalise the decision matrix from the Overview: **which populations are mandated** (documents and/or Outlook), the **default label** (§2.6 — commonly `General`, non-encrypting), whether **downgrade justification** is required, and **which labels recommend** themselves (with the prompt message).

2. Confirm the pilot ring (the group backing `LabelPolicy – Pilot – Ring1`) and the per-app friction decision (§2.7 — documents mandatory, Outlook relaxed is the common starting point).

3. **Gate:** a signed-off matrix exists — populations, mandatory scope, default label, downgrade rule, recommended labels + prompt text — before any policy setting is changed.

Step 3 — Create (or select) the pilot label policy (Configure)

1. If a policy scoped to the pilot ring does not already exist, create it: `IPMANREC-PolicyConfig.ps1 -CreatePilotPolicy -PolicyName "LabelPolicy – Pilot – Ring1" -PilotGroup "grp-labeling-pilot@contoso.com" -WhatIf`, review the `New-LabelPolicy` it would run, then re-run with `-Commit`. (If the pilot ring is already covered by an existing policy, use that policy's name instead and skip creation.)

2. Confirm in Purview → **Information Protection** → **Label policies** that the policy publishes the intended labels to the pilot group only.

3. **Gate:** a label policy targeting **only** the pilot ring exists and publishes the intended labels; non-pilot users are unaffected. Rollback: `Remove-LabelPolicy "LabelPolicy – Pilot – Ring1"`.

Step 4 — Set the default label on the pilot policy (Configure, -WhatIf first)

1. Run `IPMANREC-PolicyConfig.ps1 -SetDefault -PolicyName "LabelPolicy – Pilot – Ring1" -DefaultLabel "General" -WhatIf`. It shows how it would set the policy's **default label** for documents/email (and, via `OutlookDefaultLabel`, the Outlook default) — **without making changes**. Confirm `General` is **non-encrypting** (§2.6).

2. Re-run with `-Commit`. In the portal, the policy's **Policy settings** page now shows a default label; new documents/email for pilot users start at `General`.

3. **Gate:** `Get-LabelPolicy "LabelPolicy – Pilot – Ring1"` shows the default-label setting; a pilot user's new document defaults to `General` (allow client sync time).

Step 5 — Enable mandatory labeling on the pilot policy (Configure, -WhatIf first)

1. Decide the per-app scope (§2.7). Run `IPMANREC-PolicyConfig.ps1 -SetMandatory -PolicyName "LabelPolicy – Pilot – Ring1" -RelaxOutlook -WhatIf`. It shows the advanced settings it would apply: `Mandatory = "true"` (require a label) and, with `-RelaxOutlook`, `DisableMandatoryInOutlook = "true"` (do not block email send). **No changes are made in `-WhatIf`.**

2. Re-run with `-Commit`. Pilot users are now prompted to apply a label before saving a document (and, unless relaxed, before sending email).

3. **Gate:** a pilot user is prompted for a label on save in Word/Excel/PowerPoint; if `-RelaxOutlook` was used, email send is **not** blocked. Rollback: re-run with `-DisableMandatory -Commit` to remove the requirement.

Step 6 — Require downgrade justification on the pilot policy (Configure, -WhatIf first)

1. Run `IPMANREC-PolicyConfig.ps1 -SetDowngradeJustification -PolicyName "LabelPolicy – Pilot – Ring1" -WhatIf`, review, then `-Commit`. This turns on the policy setting requiring a **justification when a user removes a label or lowers its classification**. (In the portal this is the **Policy settings** → **"Users must provide justification…"** toggle — **verify the exact advanced-setting key in your tenant**; the script flags it.)

2. **Gate:** a pilot user who lowers a label (e.g. `Confidential` → `General`) or removes it is prompted for a justification, and the choice is recorded (visible later in audit).

Step 7 — Configure recommended-label prompts on the label (Configure, -WhatIf first)

1. Recommended labeling is set on the **label**, not the policy. Run `IPMANREC-PolicyConfig.ps1 -SetRecommend -LabelName "Confidential \ All Employees" -PromptText "This content looks sensitive – apply Confidential to protect it." -WhatIf`. It shows how it would set the label's **auto-labeling for Office apps** to **Recommend** (not Automatic) with the policy-tip message, keyed to the label's conditions. Review, then `-Commit`.

2. In the portal (**Information Protection** → **Sensitivity labels** → **[label]** → **Auto-labeling for Office apps**) confirm the mode is **Recommend** and the user message is set. **Verify your licensing/tenant** — advanced condition types are E5-tier (§2.2).

3. **Gate:** a pilot user editing content that matches the label's conditions sees a **recommend** prompt with the message and can **accept or dismiss** it (it does not force-apply).

Step 8 — Validate friction and effective settings on the pilot (Validate)

1. Run `IPMANREC-Validation.ps1 -PolicyName "LabelPolicy – Pilot – Ring1"` (read-only). It confirms the policy's **effective** advanced settings match the design: mandatory on, default = `General`, downgrade justification on, Outlook relaxed if intended, and the recommend label's mode.

2. Complete the **Testing Guide (§5)** on the pilot ring and gather pilot feedback (ticket volume, send-blocking complaints, mislabel rate).

3. **Gate:** effective settings match the matrix; pilot friction is acceptable to the governance owner. **Do not expand** until this gate passes — this is the equivalent of a simulation sign-off.

Step 9 — Expand the settings to the broad policy (Expand)

1. Apply the **same, signed-off** settings to the broad policy: run each `IPMANREC-PolicyConfig.ps1` mode from Steps 4–6 again with `-PolicyName "LabelPolicy – Broad – AllUsers"` (`-WhatIf` first, then `-Commit`). Recommended-label settings (Step 7) are on the label and already apply everywhere the label is published — no repeat needed.

2. If pilot feedback showed Outlook mandatory was too disruptive, keep Outlook relaxed broadly (`-RelaxOutlook`); tighten later only if adoption data supports it.

3. **Gate:** the broad policy's effective settings match the pilot's accepted design (`IPMANREC-Validation.ps1 -PolicyName "LabelPolicy – Broad – AllUsers"`); a rollback command is documented per setting.

Step 10 — Produce the posture evidence and operate (Validate → Operate)

1. Run `IPMANREC-Reporting.ps1 -CsvFolder <path>` to export the **mandatory-vs-recommended posture matrix** (which policies mandate, the default label per policy, downgrade state, which labels recommend) as evidence for the governance owner.

2. Run `IPMANREC-Health.ps1` on a cadence to catch **conflicting settings across overlapping policies**, inconsistent default labels, or a pilot policy left mandating after the ring dissolved.

3. **Gate:** the posture report matches the signed-off matrix; no policy conflicts flagged; the rollout is documented and reversible.

Order of operations

`Assess posture → Design matrix → Create/select pilot policy → Set default label → Enable mandatory (per-app) → Require downgrade justification → Configure recommend prompts on labels → Validate friction on pilot → Expand to broad policy → Report + operate`

5. Testing & Validation Guide — Mandatory vs Recommended Labeling Policy Design

Run these tests after the Implementation Guide, **in order**. Each has an explicit **pass/fail** signal and a rollback. Do not expand from the pilot ring to the broad policy until the relevant tests pass — because label-policy settings have **no simulation mode**, these pilot tests **are** your safety gate. All checks use the Purview portal and Security & Compliance PowerShell — the provided scripts automate the read-only ones.

> ***Reminder:*** *you are validating that the configuration *behaves as designed* (mandatory/default/downgrade/recommend), not deciding which populations should be mandated — that is the governance determination from §2.*

Test 1 — Effective policy settings match the design

• **How:** `IPMANREC-Validation.ps1 -PolicyName "LabelPolicy – Pilot – Ring1"`, or `Get-LabelPolicy "LabelPolicy – Pilot – Ring1" | Select-Object -ExpandProperty Settings`.

• **Pass:** the policy shows the intended default label (`General`), `Mandatory = true`, downgrade justification on, and `DisableMandatoryInOutlook` set if Outlook was relaxed.

• **Fail:** a setting is missing or wrong → re-run the relevant `IPMANREC-PolicyConfig.ps1` mode with `-Commit` and re-validate.

Test 2 — Default label applies to new content (pilot)

• **How:** as a **pilot** user, create a new document in Word/Excel/PowerPoint and a new email in Outlook.

• **Pass:** the new document/email is pre-set to `General` (the default label) rather than "no label"; the default is a **non-encrypting** label.

• **Fail:** no default applied (allow client sync time) or an encrypting label defaulted (wrong default — fix in Step 4) → re-check the policy default setting.

Test 3 — Mandatory labeling prompts on documents

• **How:** as a pilot user, create a document, remove any label, and attempt to **save**.

• **Pass:** the user is **required** to choose a label before the save completes (mandatory is enforcing on documents).

• **Fail:** save completes with no label → confirm `Mandatory = true` on the pilot policy and allow client sync; confirm the user is in the pilot group.

Test 4 — Outlook friction matches the per-app decision

• **How:** as a pilot user, compose and **send** an email with no label.

• **Pass (relaxed):** if `-RelaxOutlook` was used, send is **not** blocked (mandatory does not apply in Outlook). **Pass (strict):** if Outlook was intentionally kept mandatory, send is blocked until a label is chosen.

• **Fail:** behaviour is the opposite of the design → re-check `DisableMandatoryInOutlook` on the policy (Step 5); this is the most common adoption complaint, so confirm it matches the signed-off decision.

Test 5 — Downgrade requires a justification and records it

• **How:** as a pilot user, take a document labelled `Confidential \ All Employees` and change it to `General` (or remove the label).

• **Pass:** the user is prompted to type a **justification**; the downgrade only completes after one is supplied.

• **Fail:** downgrade completes silently → confirm the downgrade-justification setting on the policy (Step 6); **verify the advanced-setting key in your tenant** if the portal toggle and PowerShell disagree.

Test 6 — Downgrade justification appears in audit (evidence)

• **How:** after Test 5, in Purview → **Audit** (or `Search-UnifiedAuditLog`), search for the label-change / justification activity for the test document.

• **Pass:** the downgrade event (old label → new label, with the justification) is discoverable in the audit log — the intended evidence trail exists.

• **Fail:** no audit event → confirm auditing is enabled for the tenant and allow audit-ingestion latency; **verify audit availability for your licensing/tenant.**

Test 7 — Recommended prompt suggests (does not force) the label

• **How:** as a pilot user, create content that matches `Confidential \ All Employees` conditions (e.g. the configured sensitive info type).

• **Pass:** a **recommend** prompt appears with the configured message ("This content looks sensitive — apply Confidential to protect it."); the user can **accept** (label applied) **or dismiss** (no label forced).

• **Fail (forces the label):** the label is auto-applied without a choice → the label's Office-app auto-labeling is set to **Automatic**, not **Recommend** — fix in Step 7. **Fail (no prompt):** conditions not matching or feature not licensed → verify the condition and **your licensing/tenant.**

Test 8 — No conflicting settings across overlapping policies

• **How:** `IPMANREC-Health.ps1` (read-only).

• **Pass:** no user is covered by two policies that set **different** mandatory/default values; policy priority/order resolves predictably; the pilot policy targets only the pilot ring.

• **Fail:** a conflict is flagged (e.g. two policies with different default labels hitting the same users) → resolve the overlap (priority or scope) before expanding.

Test 9 — Broad-policy settings match the accepted pilot design

• **How:** after Step 9, `IPMANREC-Validation.ps1 -PolicyName "LabelPolicy – Broad – AllUsers"`.

• **Pass:** the broad policy's effective settings equal the **accepted** pilot settings (mandatory scope, default label, downgrade, Outlook relaxation).

• **Fail:** divergence between pilot and broad → re-apply the signed-off settings to the broad policy and re-validate before relying on it.

Test 10 — Posture evidence is complete

• **How:** `IPMANREC-Reporting.ps1 -CsvFolder <path>`; review the exported matrix.

• **Pass:** the report lists every policy's mandatory state, default label, downgrade state, and every label's recommend/automatic mode — matching the signed-off design matrix.

• **Fail:** a gap or mismatch → reconcile the configuration with the matrix and re-export.

Exit criteria (all must be true before broad enforcement)

`Effective settings match design · default label applies · mandatory prompts on documents · Outlook friction matches decision · downgrade requires + records justification · recommend suggests (never forces) · no policy conflicts · broad == accepted pilot · posture evidence complete · rollback documented per setting`

6. Troubleshooting Guide — Mandatory vs Recommended Labeling Policy Design (Microsoft Purview)

Symptom → likely root cause → fix. Use `IPMANREC-Discovery.ps1` (current posture), `IPMANREC-Validation.ps1` (a policy's effective settings), `IPMANREC-Health.ps1` (conflicts / pilot

sanity), and `IPMANREC-Reporting.ps1` (posture matrix) to confirm before and after each fix. Label-policy behaviour settings have **no simulation mode**, so most issues surface on the **pilot ring first** — that is by design. Advanced-setting **key names differ by tenant/version**; where a key is flagged, **verify it in your tenant** (portal toggle vs PowerShell) before a broad change.

A. Availability / setup

A setting or toggle isn't present when configuring the policy

• **Licensing insufficient.** Mandatory labeling and default labels work broadly, but **recommended / automatic labeling for Office apps **(client-side conditions) is plan-gated (typically E5 / E5 Compliance).** Fix: **confirm the subscription in the M365 admin center; the recommend prompt (Step 7) needs the advanced classification licensing —** verify your licensing/tenant** (Req 2.1).

• **Wrong role.** Label/policy edits require **Information Protection Admin** (or a role that includes it).

Fix: confirm the role under Purview → Roles & scopes (Req 2.2).

Portal toggle and PowerShell advanced-setting disagree

• **Key name drift.** The exact advanced-setting key (e.g. downgrade justification) can differ by tenant/version. **Fix:** set the behaviour once in the **portal Policy-settings wizard**, then read it back with `IPMANREC-Validation.ps1` to learn the resolved key your tenant uses; align the script value to that key. The scripts flag these keys as "verify in your tenant".

B. Behaviour doesn't apply (false negatives)

The default label doesn't apply to new content

• **Client hasn't synced.** Policy changes take time to reach Office clients. **Fix:** wait for client sync (sign out/in or restart Office); re-test with a pilot user.

• **User not in the policy scope.** The pilot user isn't in the published group. **Fix:** `IPMANREC-Discovery.ps1` shows the policy's scope; confirm the user is in `grp-labeling-pilot@contoso.com`.

• **Default set on the wrong policy or as "None". Fix:** `IPMANREC-Validation.ps1 -PolicyName …`; confirm `DefaultLabelId` resolves to the intended **non-encrypting** label (Step 4).

Mandatory labeling doesn't prompt on save

• `Mandatory` **not effective / client not synced. Fix:** confirm `Mandatory = true` on the pilot policy with `IPMANREC-Validation.ps1`; allow client sync; confirm the user is in the pilot group.

The recommend prompt never appears

• **Conditions don't match.** The label's Office-app auto-labeling conditions (e.g. the sensitive info type) aren't met by the test content. **Fix:** test with content that clearly matches; lower confidence if too strict.

• **Feature not licensed.** Client-side recommend conditions are licensing-gated. **Fix:** **verify your licensing/tenant** (Req 2.1).

C. Behaviour is too aggressive (false positives / adoption pain)

Outlook blocks email send and users complain

• **Outlook kept strict when it should be relaxed.** This is the single most common adoption complaint. **Fix:** confirm the per-app decision (§2.7). If email should not be blocked, run `IPMANREC-PolicyConfig.ps1 -SetMandatory -RelaxOutlook -Commit` (`DisableMandatoryInOutlook = true`); re-test send. If email *should* be mandatory, this is working as designed — confirm the signed-off decision.

The recommend label force-applies instead of suggesting

• **Label set to Automatic, not Recommend. Fix:** `IPMANREC-Validation.ps1 -RecommendLabel …` shows the mode; re-run `IPMANREC-PolicyConfig.ps1 -SetRecommend -LabelName …` so auto-labeling is **Recommend** (a prompt the user can dismiss), not Automatic (Step 7).

A default label unexpectedly encrypts new content

• **Encrypting label chosen as the default.** A default that applies encryption causes surprise encryption and breaks sharing. **Fix:** set the default to a **non-encrypting** label (e.g. `General`); rollback is `DefaultLabelId = None`. Confirm with `IPMANREC-Discovery.ps1`.

D. Downgrade justification

Downgrade completes silently (no justification prompt)

• **Setting off or wrong key. Fix:** re-run `IPMANREC-PolicyConfig.ps1 -SetDowngradeJustification -Commit`; if it still doesn't prompt, **verify the advanced-setting key in your tenant** (portal toggle "Users must provide justification…") and align the script to the resolved key.

The justification isn't discoverable as evidence

• **Auditing off or latency. Fix:** confirm unified audit is enabled for the tenant; allow audit ingestion latency, then re-search (Test 6). **Verify audit availability for your licensing/tenant.**

E. Conflicting / overlapping policies

Users get unpredictable mandatory/default behaviour

• **Overlapping policies with different values.** Priority resolves opaquely. **Fix:** `IPMANREC-Health.ps1` flags policies with different mandatory/default values hitting overlapping populations; reconcile via **one coherent policy per population** or an explicit, documented priority (§7).

A pilot policy is still mandating after the ring dissolved

• **Stale pilot. Fix:** `IPMANREC-Health.ps1` flags pilot-named policies still mandating; confirm the design expanded to the broad policy, then retire the pilot policy.

F. Expansion / evidence

Broad policy behaves differently from the accepted pilot

• **Settings not re-applied identically. Fix:** re-run each `IPMANREC-PolicyConfig.ps1` mode against `LabelPolicy – Broad – AllUsers`, then `IPMANREC-Validation.ps1` on the broad policy to confirm it equals the accepted pilot design (Step 9).

The posture matrix has gaps or mismatches

• **Configuration drifted from the design matrix. Fix:** `IPMANREC-Reporting.ps1 -CsvFolder <path>`; reconcile any policy/label that doesn't match the signed-off matrix and re-export.

Escalation checklist

1. `IPMANREC-Discovery.ps1` — do the pilot/broad policies exist with the intended scope?
2. `IPMANREC-Validation.ps1 -PolicyName …` — do effective settings (mandatory / default / downgrade / Outlook / recommend) match the design on the policy in question?
1. `IPMANREC-Health.ps1` — any conflicting/overlapping policies or a stale pilot still mandating?
2. `IPMANREC-Reporting.ps1 -CsvFolder …` — does the posture matrix match the signed-off design?
3. Still stuck → capture the exact behaviour, compare the **portal toggle** to the **resolved advanced setting**, and** verify the current Microsoft guidance / your tenant** for the setting's key before a tenant-wide change. Regulatory/records determinations (which populations to mandate) remain with the governance owner — this guide fixes the *technical behaviour*, not the *policy decision*.

7. Best Practices — Mandatory vs Recommended Labeling Policy Design

Field-tested practices for deciding **where to require a label versus where to suggest one** — getting high classification rates **without a user revolt**. These complement the Implementation and Testing guides.

Choosing mandatory vs recommended

• **Mandatory forces *a* label, not the *right* one.** A hard requirement guarantees content is labelled, but under pressure users pick the first (usually lowest) label to dismiss the prompt — which quietly corrupts your classification data. Pair mandatory with a **sensible default** and **recommended prompts**

so the *easy* choice is also the *right* one.

• **Recommend where a nudge is enough.** For content that isn't uniformly sensitive, a **recommend** prompt ("this looks like financial data — apply Confidential?") raises correct-label rates without the backlash. Reserve mandatory for populations/content where an unlabelled item is a real risk.

• **Default label is your floor, not your answer.** Set the default to a **low, non-encrypting** label (commonly `General`) so nothing is ever silently unlabelled — but never default to a protected/encrypting label (surprise encryption is the fastest way to lose user trust).

• **Layer the levers.** The strongest design is usually *default `General` + recommend the right higher label on matching content + mandatory so nothing escapes unlabelled + downgrade justification for evidence* — not mandatory alone.

Rollout discipline (no simulation — so pilot is everything)

• **There is no simulation mode for these settings.** Unlike service-side auto-labeling, mandatory/default/downgrade settings apply the moment you commit them. The **pilot ring is your only safety net** — always configure on `LabelPolicy – Pilot – Ring1` first and validate friction before touching `LabelPolicy – Broad – AllUsers`.

• `-WhatIf` **before every commit.** Every provided config action previews with `-WhatIf` and changes nothing until `-Commit`. Keep it that way; read the preview.

• **Relax Outlook first, tighten later.** Blocking every email send is the loudest adoption complaint. Start with mandatory on **documents**, Outlook **relaxed** (`DisableMandatoryInOutlook`), and only tighten Outlook if adoption data shows it's needed.

• **Measure pilot friction, not just correctness.** Track ticket volume, send-blocking complaints, and mislabel rate on the pilot. Acceptable friction — not just "the setting works" — is the gate to expand.

Defaults, downgrade, and evidence

• **Downgrade justification is cheap and high-value.** Requiring a typed justification to lower/remove a label deters careless downgrades and produces an **audit trail** for investigations — turn it on early.

• **Confirm the default classification with governance.** The enterprise default label is a real business decision (§2.6), not a technical default — get it signed off; the wrong default over- or under-classifies everything.

• **Keep the default non-encrypting.** A default that encrypts will encrypt content users never intended to protect. Default to marking-only tiers.

Avoiding policy conflicts

• **One coherent policy per population.** Overlapping label policies with **different** mandatory/default values create unpredictable behaviour (priority decides, opaquely). Run `IPMANREC-Health.ps1` to catch users covered by conflicting policies.

• **Mind policy priority.** When users are in scope of multiple policies, order/priority resolves which default and mandatory value wins — verify it resolves the way you intend, don't assume.

• **Retire the pilot policy cleanly.** Once broad rollout is validated, decide whether the pilot policy stays (as a fast ring for future changes) or is removed — don't leave an orphaned pilot policy mandating for a group that no longer exists.

Program & change management

• **Communicate before you mandate.** The ticket flood comes from surprise. Tell users what will change (the prompt on save, the justification on downgrade), why, and where to get help — before you commit the broad policy.

• **Document a rollback per setting.** Each setting is reversible: remove mandatory (`-DisableMandatory`), unset the default, unpublish the policy (`Remove-LabelPolicy`), or set a recommend label back from Automatic. Know the exact command before you commit.

• **Re-baseline periodically.** Run `IPMANREC-Discovery.ps1` / `IPMANREC-Reporting.ps1` on a cadence to catch policy drift, conflicting defaults, and labels silently flipped from Recommend to Automatic.

Security & safety hygiene

• **No credentials in code.** All scripts authenticate interactively via `Connect-IPPSSession`; never embed secrets. Store any exported evidence securely.

• **Read-only or `-WhatIf` by default.** Discovery/validation/reporting/health scripts are read-only; the config script defaults to `-WhatIf` and requires `-Commit` for any write. Keep it that way.

• **Verify licensing/tenant before relying on recommended prompts.** Client-side recommend labeling with advanced conditions is plan-gated and the portal evolves — flag, don't assume.

8. Deliverables Summary — Mandatory vs Recommended Labeling Policy Design

What you receive

A complete, self-guided package to **decide and configure where a sensitivity label is required versus suggested** in Microsoft Purview — mandatory labeling, default labels, downgrade justification, and recommended (prompt) labeling — piloted on a ring, validated, then expanded, using only the Microsoft Purview portal and Security & Compliance PowerShell.

Documentation

Document	What it gives you
Solution Overview	The four policy levers (mandatory, default, downgrade justification, recommended prompts), the mandatory-vs-recommended decision matrix, the naming convention, and the deployment model.

Document	What it gives you
Requirements Guide	Solution-specific prerequisites (an existing label taxonomy, licensing for recommended labeling, Information Protection role, S&C PowerShell, a pilot ring, an agreed default classification and downgrade rule, and the per-app friction decision) on top of the Purview Global Requirements.
Implementation Guide	Ordered steps: assess posture → design the matrix → create/select the pilot policy → set the default label → enable mandatory (per-app) → require downgrade justification → configure recommend prompts on labels → validate friction → expand to the broad policy → report. Each step named, gated, and reversible; `-WhatIf` before every commit; pilot-first (no simulation mode).
Testing & Validation Guide	10 pass/fail tests — effective settings, default application, mandatory prompts, per-app Outlook friction, downgrade justification + audit evidence, recommend (never force), policy-conflict check, and broad == accepted pilot.
Best Practices	Choosing mandatory vs recommended, no-simulation rollout discipline, defaults/downgrade/evidence, avoiding policy conflicts, program/change-management, and safety guidance.
Troubleshooting Guide	The common failure modes (mandatory not prompting, Outlook blocking send, default not applying, recommend forcing instead of suggesting, downgrade silent, policy conflicts) with resolutions.

PowerShell tools (5)

All run through `Connect-IPPSSession` (Security & Compliance PowerShell); read-only or `-WhatIf` by default, writes gated behind `-Commit`. No credentials in code.

Script	Role	Default
`IPMANREC-Discovery.ps1`	**Posture assessment** — inventory label policies, each policy's advanced settings (mandatory / default / Outlook) and scope, and each label's Office-app auto-labeling mode (Automatic vs Recommend).	Read-only

Script	Role	Default
`IPMANREC-PolicyConfig.ps1`	**Configuration / remediation helper** — create the pilot policy, set the default label, enable/disable mandatory labeling (with per-app Outlook relaxation), require downgrade justification, and set a label to **Recommend** with a prompt message.	`-WhatIf`
`IPMANREC-Validation.ps1`	**Effective-settings validation** — confirm a policy's resolved advanced settings (mandatory, default, downgrade, Outlook) and a label's recommend mode match the signed-off design.	Read-only
`IPMANREC-Reporting.ps1`	**Posture matrix / evidence** — export the mandatory-vs-recommended matrix across all policies and labels (default label per policy, mandatory + downgrade state, recommend/automatic per label) to CSV.	Read-only
`IPMANREC-Health.ps1`	**Scope/health helper** — flag conflicting settings across overlapping policies, inconsistent default labels, policy-priority resolution, and pilot-scope sanity.	Read-only

Outcomes

- A signed-off **mandatory-vs-recommended matrix** configured in the tenant — mandatory where risk justifies friction, recommended where a nudge is enough.
- A **default label** so no new content is silently unclassified, and **downgrade justification** producing an audit trail.
- **Recommended-label prompts** that suggest the right label without forcing it.
- Settings **piloted on a ring** and validated for friction before broad expansion (the safe substitute for a non-existent simulation mode).
- Repeatable read-only tooling to report the posture and catch policy conflicts over time.

Explicitly out of scope (deferred)

• **The label taxonomy, encryption, content marking, and retention labels** — assumed to exist (see the taxonomy/labels solutions); this solution designs the *policy behaviour* over them.

• **Service-side auto-labeling at scale, DLP policy authoring, insider risk, and communication compliance** — separate Purview solutions.

• **Which populations *should* be mandated, the enterprise default classification, and downgrade rules** — information-governance determinations owned by compliance/security/records; this solution provides the configuration method and evidence.

• Any capability your **licensing/tenant** does not include — verify before relying on recommended (client-side) labeling with advanced conditions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPMANREC-Discovery.ps1

IPMANREC-Health.ps1

IPMANREC-PolicyConfig.ps1

IPMANREC-Reporting.ps1

IPMANREC-Validation.ps1